

PROYECTO FINAL • PLATAFORMA FULL-STACK

Bluebook

MARKETPLACE Y VALUACIÓN DE AUTOS USADOS

El precio, legible: cada auto se muestra junto a lo que el modelo cree que vale — para que el trato nunca se adivine.

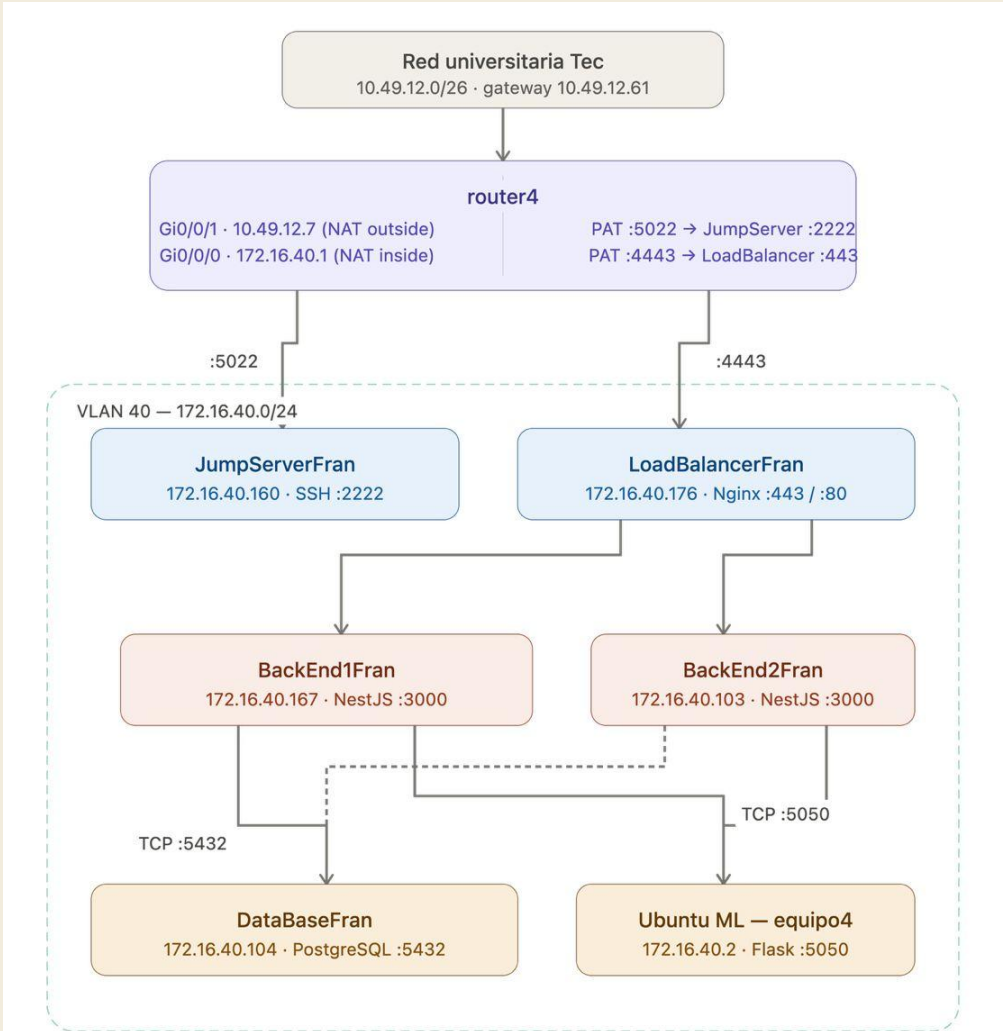
Del precio a ciegas al *veredicto* visible

- 01 Problema** – comprar o vender un auto usado a ciegas: ¿el precio es justo?
- 02 Solución** – un random forest valúa cada vehículo y el marketplace expone ese veredicto de forma visual.
- 03 Dos experiencias** – integradas sobre la misma valuación:
 - **Studio:** valúa tu auto (specs → estimado + rango + drivers + depreciación).
 - **Marketplace:** publica, navega, recibe recomendaciones y ve el “deal”.

Decisiones de arquitectura *clave*

- 01 Gateway único** – El frontend nunca habla ni con la base de datos ni con el modelo, sino que todo pasa por los backends. Es decir, si quieres algún tipo de dato, necesita pasar por el backend.
- 02 URLs relativas + proxy de Vite** – El frontend no tiene escrita la IP del backend en ningún lado. Solo dice /listings, sin dominio y sin IP. En desarrollo, Vite intercepta ese request y lo manda al backend local. En producción, Nginx hace exactamente lo mismo.
- 03 Valuation-on-write** – la valuación se calcula al escribir el listing. Cuando el marketplace carga, no hay que llamar al modelo. De esta manera, las páginas cargan mucho más rápido.

Decisiones de arquitectura *clave*



Reglas de firewall	
JumpServerFran	TCP 2222 · ESTABLISHED · loopback
LoadBalancerFran	TCP 443 · TCP 80 · TCP 2222
BackEnd1 / BackEnd2	TCP 3000 solo desde 172.16.40.176
DataBaseFran	TCP 5432 solo desde BackEnd1 y BackEnd2
ML service (ufw)	TCP 5050 desde .167 (B1) y .103 (B2)

☐ Gateway / acceso ☐ Capa de aplicación ☐ Datos / servicios ☐ Router / NAT

Stack *tecnológico*

01 FRONTEND

React 19 • TypeScript • Vite 8 • React Router 7 • Vitest + Testing Library.

02 BACKEND

NestJS 11 • Prisma 6 • PostgreSQL 16 • Passport-JWT • argon2 • Helmet • Throttler • Swagger.

03 MODELO

Python • Flask • scikit-learn (random forest) • pandas / NumPy.

04 INFRAESTRUCTURA

Local: Docker Compose (Postgres + backend + model-service)

Producción: OpenStack VMs — PM2 (NestJS), systemd (modelo), Postgres nativo

Identidad y *contraseñas*

- 01 **AuthModule** – register · login · verify-email · refresh · logout.
- 02 **Hashing argon2id** – contraseñas con hashing memory-hard, resistente a GPU; la contraseña en crudo nunca toca disco.
- 03 **passwordHash nunca se expone** – UserService.toPublic() lo elimina de toda respuesta de la API.
- 04 **Email verificado obligatorio** – antes de acceder a rutas protegidas o publicar un listing.

Sesiones, tokens y *rotación*

- 01 Access token** – JWT de TTL corto (15 min) en el cuerpo de la respuesta; el frontend lo guarda solo en memoria – anti-XSS, nunca en localStorage.
- 02 Refresh token** – opaco de 32 bytes en cookie httpOnly/Secure (scope /auth); la BD guarda solo su hash SHA-256 → un dump de la BD no sirve.
- 03 Rotación + detección de reuse** – presentar un token ya revocado invalida toda la familia (familyId) y fuerza re-login.
- 04 Restauración silenciosa** – sesión recuperada al arrancar + interceptor 401 con single-flight refresh.

Defensa en profundidad y *auditoría*

- 01 TLS / HTTPS** – Proxy - nginx (tras un ingress).
- 02 Encriptación at-rest** – AES-256-GCM autenticada (IV aleatorio por llamada, auth tag) vía CryptoService global.
- 03 Headers de seguridad (Helmet)** – CSP, nosniff, X-Frame-Options, Referrer-Policy, Permissions-Policy.
- 04 Rate limiting (Throttler)** – 200 req/60 s global · 5/60 s en login – defensa anti fuerza bruta.
- 05 Audit log append-only** – auth_audit_log registra login ok/fallido, register, logout, rotación, reuse, verificación y cambios de rol; sobrevive al borrado del usuario (ON DELETE SET NULL).

Módulos NestJS y capas *transversales*

01 – MÓDULOS POR DOMINIO



02 Capas globales – guards (JWT por defecto, roles, verified), ValidationPipe con whitelist – el cliente no puede setear campos derivados –, Throttler, Helmet y Swagger en /docs.

03 Model client – ModelModule es el único que habla con Flask (server-to-server, por la red interna).

Base de datos y *normalización*

- 01 Postgres + Prisma** – users, refresh_tokens, email_verification_tokens, listings, listing_price_history, favorites, search_history, auth_audit_log.
- 02 Normalización 1NF → 3NF** – con justificación; PKs cuid (no secuenciales, sin contención de lock).
- 03 Excepción documentada** – Listing cachea valores derivados del modelo (predictedValue/low/high, dealDeltaPct): denormalización controlada, escrita solo por ListingsService en una transacción.
- 04 FKs** – ON DELETE CASCADE para datos del usuario y SET NULL para auditoría e históricos que deben sobrevivir.

Valuación, históricos y *recomendaciones*

- 01 Valuation-on-write** – al crear o editar un listing se llama a `/predict` y se guardan `predicted*`, `dealDeltaPct` y el deal badge (Under / Fair / Over, umbral $\pm 10\%$), todo en una transacción junto al historial.
- 02 Históricos append-only** – `listing_price_history` (una fila por cada cambio de precio o revaluación) y `search_history` (filtros como JSONB, señal de preferencia).
- 03 Recomendaciones • GET /recommendations** – combinan `deal_score` + `preference_score` (favoritos + historial); con cold-start caen a solo deal; cada ítem trae su «why».
- 04 Calidad** – ~196 tests Jest en el backend; seed idempotente y reproducible.

Marketplace y *detalle*

- 01 Marketplace** – grid de ListingCard + barra de filtros (maker, tipo, estado, banda de precio) + orden + paginación.
- 02 Deal badge legible** – el precio de venta se muestra junto al valor del modelo, con un badge Under / Fair / Over y el % de delta.
- 03 Riel «Recommended for you»** – consume /recommendations y muestra el «why» de cada sugerencia.
- 04 Detalle del listing • /listings/:id** – análisis visual sobre/infra (asking vs. banda del modelo), veredicto del deal, historial de precios, contacto y favorito.

Studio, Sell y veredictos *accionables*

- 01 Studio** – wizard de appraisal (una pregunta por tarjeta → estimado + rango + depreciación + drivers); garage y comparación head-to-head.
- 02 Puente "List this car →"** – lleva una valuación guardada a un formulario Sell pre-llenado.
- 03 My Listings** – crear / editar / borrar / cambiar status (draft • active • sold); publicar como active exige email verificado.
- 04 Veredicto según intención** – (lib/deal.ts) el mismo precio se narra distinto para comprador y vendedor, con guía accionable: oferta de apertura / valor justo / techo.

Accesibilidad y *experiencia*

- 01 No solo color** – el deal badge usa texto + símbolo + color (tres canales redundantes) + aria-label con el veredicto completo.
- 02 Estructura semántica** – skip link a <main>, <nav aria-label>, listings como <article> con encabezados.
- 03 Teclado** – tarjetas operables con Enter / Espacio (role="button", tabIndex=0) y foco visible.
- 04 Movimiento y contraste** – prefers-reduced-motion respetado; contraste WCAG AA en toda la interfaz.
- 05 Degradación elegante** – si /options no responde, cae a mocks y la UI sigue usable; estados de carga / vacío / error en todas las vistas.

Hacia dónde *sigue*

- 01 Encriptación a nivel de columna** – del contacto del seller usando el CryptoService ya disponible, sin migración de schema.
- 02 Recomendaciones más ricas** – incorporar más señales (vistas, tiempo en detalle) y explicaciones más finas del "why".
- 03 Tracking anónimo** – de búsquedas (el schema ya admite userId nulo) para recomendaciones pre-login.
- 04 Escalado del model-service** – al ser stateless, escalar horizontalmente y versionar el modelo; reentrenos automatizados.
- 05 Observabilidad** – métricas y dashboards sobre el audit log y las latencias del modelo.

Cobertura

#	REQUISITO	DÓNDE
1	Autenticación	AuthModule + auth UI
2	Base de datos	Prisma + migraciones
3	Seguridad • tokens • encriptción	argon2, JWT, cookies, TLS, AES
4	Históricos	price_history, search_history, audit_log
5	Manejo de sesiones	rotación + silent refresh
6	UI/UX • accesibilidad	a11y, teclado, contraste, no-solo-color
7	Formas normales	normalización a 3NF
8	Diseño de BD + justificación	ER + documento de diseño

Demo en vivo → login • marketplace • recomendaciones • detalle • valuación.

Gracias. *¿Preguntas?*